AI Assistant Coding
Lab - 9.3

Task 1: Basic Docstring Generation
Scenario
You are developing a utility function that processes numerical lists and must be properly documented for future maintenance.
Requirements
• Write a Python function to return the sum of even numbers and sum of odd numbers in a given list
• Manually add a Google Style docstring to the function
• Use an AI-assisted tool (Copilot / Cursor AI) to generate a function-level docstring
• Compare the AI-generated docstring with the manually written docstring
• Analyze clarity, correctness, and completeness
Expected Output
• Python function with manual Google-style docstring
• AI-generated docstring for the same function
• Comparison explaining differences between manual and AI-generated documentation
• Improved understanding of AI-generated function-level documentation

Prompt

```
#Generate a detailed Google-style docstring for the Python function that calculates the sum of even and odd numbers in a list, including Args, Returns, and Example sections. Improve clarity, readability, and ensure proper type descriptions following standard Google documentation conventions.
```

Code

```
def sum_even_odd(numbers):
    """
    Calculates the sum of even and odd numbers in a given list.

    This function takes a list of integers as input and computes the sum of all even numbers and the sum of all odd numbers separately. It returns a tuple containing both sums.

    Args:
        numbers (list of int): A list of integers to be processed.
    Returns:
        tuple: A tuple containing two elements:
            - sum_even (int): The sum of all even numbers in the input list.
            - sum_odd (int): The sum of all odd numbers in the input list.
    Example:
        >>> sum_even_odd([1, 2, 3, 4, 5])
        (6, 9)
        >>> sum_even_odd([10, 15, 20, 25])
        (30, 40)
```

```
    """
    sum_even = 0
    sum_odd = 0

    for num in numbers:
        if num % 2 == 0:
            sum_even += num
        else:
            sum_odd += num

    return sum_even, sum_odd
# Example usage
if __name__ == "__main__":
    even_sum, odd_sum = sum_even_odd([1, 2, 3, 4, 5])
    print(f"Sum of even numbers: {even_sum}, Sum of odd numbers: {odd_sum}")
```

Output

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab9.1.py"
Sum of even numbers: 6, Sum of odd numbers: 9
PS C:\Users\dasar\AI-Assistant-Coding> & C:/Users/dasar/AI-Assistant-Coding/.venv/Scripts/Activate.ps1
(.venv) PS C:\Users\dasar\AI-Assistant-Coding>
```

Task 2: Automatic Inline Comments
Scenario
You are developing a student management module that must be easy to understand for new developers.
Requirements
• Write a Python program for an sru_student class with the following:
– Attributes: name, roll_no, hostel_status
– Methods: fee_update() and display_details()
• Manually write inline comments for each line or logical block
• Use an AI-assisted tool to automatically add inline comments
• Compare manual comments with AI-generated comments
• Identify missing, redundant, or incorrect AI comments
Expected Output
• Python class with manually written inline comments
• AI-generated inline comments added to the same code
• Comparative analysis of manual vs AI comments
• Critical discussion on strengths and limitations of AI-generated comments

Prompt

```
#Add meaningful and professional inline comments to the student management class explaining attributes, methods, and logical blocks clearly. Ensure comments are concise, non-redundant, and improve overall code readability.
```

Code

```
class Student:
    def __init__(self, name, age, grade):
        """
        Initializes a new instance of the Student class.

        Args:
            name (str): The name of the student.
            age (int): The age of the student.
            grade (str): The grade of the student (e.g., 'A', 'B', 'C').
        """
        self.name = name  # Store the student's name
        self.age = age    # Store the student's age
        self.grade = grade  # Store the student's grade

    def is_passing(self):
        """
        Determines if the student is passing based on their grade.

        Returns:
            bool: True if the student's grade is 'A', 'B', or 'C'; False otherwise.
        """
        # Check if the grade is one of the passing grades
        return self.grade in ['A', 'B', 'C']

    def __str__(self):
        """
        Provides a string representation of the Student instance.

        Returns:
            str: A string containing the student's name, age, and grade.
        """
        return f"Student(name={self.name}, age={self.age}, grade={self.grade})"
# Example usage
if __name__ == "__main__":
    student1 = Student("Alice", 20, 'A')
    student2 = Student("Bob", 22, 'D')

    print(student1)  # Output: Student(name=Alice, age=20, grade=A)
```

```
    print(student2)   # Output: Student(name=Bob, age=22, grade=D)

    print(f"{student1.name} is passing: {student1.is_passing()}")   # Output:
Alice is passing: True
    print(f"{student2.name} is passing: {student2.is_passing()}")   # Output:
Bob is passing: False
```

Output

```
(.venv) PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab9.1.py"
Student(name=Alice, age=20, grade=A)
Student(name=Bob, age=22, grade=D)
Alice is passing: True
Bob is passing: False
(.venv) PS C:\Users\dasar\AI-Assistant-Coding>
```

Task 3: Module-Level and Function-Level Documentation
Scenario
You are building a small calculator module that will be shared across multiple projects and requires structured documentation.
Requirements
• Write a Python script containing 3–4 functions (e.g., add, subtract, multiply, divide)
• Manually write NumPy Style docstrings for each function
• Use AI assistance to generate:
– A module-level docstring
– Individual function-level docstrings
• Compare AI-generated docstrings with manually written ones
• Evaluate documentation structure, accuracy, and readability
Expected Output
• Python script with manual NumPy-style docstrings
• AI-generated module-level and function-level documentation
• Comparison between AI-generated and manual documentation
• Clear understanding of structured documentation for multi-function scripts

Prompt

```
#Generate structured NumPy-style docstrings for all calculator functions and
create a professional module-level docstring describing purpose and usage.
Ensure proper documentation structure including Parameters, Returns, and
Raises sections where applicable.
```

Code

```
"""
Calculator Module
This module provides basic arithmetic functions for addition, subtraction,
multiplication, and division. Each function takes two numerical inputs and
returns the result of the respective operation. The module is designed to be
simple and easy to use for basic mathematical calculations.
Functions:
- add(a, b): Returns the sum of a and b.
```

```
- subtract(a, b): Returns the difference of a and b.
- multiply(a, b): Returns the product of a and b.
- divide(a, b): Returns the quotient of a and b. Raises a ValueError if b is
zero.
"""
def add(a, b):
    """
    Add two numbers.

    Parameters
    ----------
    a : float
        The first number to be added.
    b : float
        The second number to be added.

    Returns
    -------
    float
        The sum of a and b.
    """
    return a + b
def subtract(a, b):
    """
    Subtract two numbers.

    Parameters
    ----------
    a : float
        The first number (minuend).
    b : float
        The second number (subtrahend).

    Returns
    -------
    float
        The difference of a and b.
    """
    return a - b

def multiply(a, b):
    """
    Multiply two numbers.

    Parameters
```

```
    ----------
    a : float
        The first number to be multiplied.
    b : float
        The second number to be multiplied.

    Returns
    -------
    float
        The product of a and b.
    """
    return a * b

def divide(a, b):
    """
    Divide two numbers.

    Parameters
    ----------
    a : float
        The dividend (numerator).
    b : float
        The divisor (denominator).

    Returns
    -------
    float
        The quotient of a and b.

    Raises
    ------
    ValueError
        If b is zero, division by zero is not allowed.
    """
    if b == 0:
        raise ValueError("Cannot divide by zero.")
    return a / b
# Example usage
if __name__ == "__main__":
    print(add(5, 3))        # Output: 8
    print(subtract(5, 3))   # Output: 2
    print(multiply(5, 3))   # Output: 15
    try:
        print(divide(5, 0)) # This will raise a ValueError
    except ValueError as e:
```

```
        print(e)              # Output: Cannot divide by zero.
    print(divide(5, 2))      # Output: 2.5
```

Output